

Diagramă schemă bază de date

Inainte

```
+-----+
| departments |
+-----+

| id      : SERIAL    | (PK)
| name    : VARCHAR(100) | NOT NULL
+-----+

+-----+
| employees |
+-----+
| id : SERIAL | (PK)
| email : VARCHAR(100) | NOT NULL
| department_id : INTEGER | (FK)
| version : INTEGER | DEFAULT 0
| is_deleted : BOOLEAN | DEFAULT FALSE
| deleted_at : TIMESTAMP
| deleted_by : VARCHAR(100)
+-----+
```

CONSTRAINTS:

- fk_employee_department → departments(id)

Relations:

departments (1) ————— (N) employees

După

```
+-----+
| departments |
+-----+
```

```

+-----+
| id : SERIAL      | (PK)
| name : VARCHAR(100) | NOT NULL
+-----+

+-----+
|          employees          |
+-----+

| id : SERIAL      | (PK) |
| email      : VARCHAR(100) | NOT NULL      |
| salary      : DECIMAL(12,2) | [Modified in V4] |
| phone      : VARCHAR(20) | [Added in V2]   |
| department_id : INTEGER      | (FK)           |
| version     : INTEGER      | DEFAULT 0       |
| is_deleted   : BOOLEAN      | DEFAULT FALSE   |
| deleted_at   : TIMESTAMP                                |
| deleted_by   : VARCHAR(100)                               |
| created_at   : TIMESTAMP      | [Added in V6]   |
| created_by   : VARCHAR(100) | [Added in V6]   |
| updated_at   : TIMESTAMP      | [Added in V6]   |
| updated_by   : VARCHAR(100) | [Added in V6]   |
+-----+

```

INDEXES:

- idx_employees_department (department_id) [V5]
- idx_employees_email (email) [V6]

CONSTRAINTS:

- fk_employee_department → departments(id)

```
+-----+
|      projects      |
+-----+
| id : SERIAL      | (PK)      |
| name : VARCHAR(100) | NOT NULL  |
| description  : TEXT      |
| start_date   : DATE      |
| department_id : INTEGER  | (FK)      |
+-----+
```

CONSTRAINTS:

- fk_project_department → departments(id)

Relations:

departments (1) — (N) employees

departments (1) — (N) projects

Rularea migrărilor înainte și înapoi

Inainte

	ID	author	filename	dateexecuted	orderexecuted	exectype	md5sum
1	1	student	V1_initial.xml	2026-05-14 16:55:11.518689	1	EXECUTED	9:9b788ded8582ba451ee24b18a6d
2	2	student	V2_add_phone.xml	2026-05-14 16:55:11.524532	2	EXECUTED	9:534583f99df3b62850d0eb18850
3	3	student	V3_create_projects.xml	2026-05-14 16:55:11.532569	3	EXECUTED	9:1854251e4a110d412fe1437289f
4	4	student	V4_modify_salary.xml	2026-05-14 16:55:11.535148	4	EXECUTED	9:569dc028c44056d08f5c76799a0
5	5	student	V5_indexes.xml	2026-05-14 16:55:11.540842	5	EXECUTED	9:31659271a79e804ecf102a2b2c2
6	6	student	V6_add_audit_columns.xml	2026-05-14 16:55:11.545978	6	EXECUTED	9:319b226c32b2f575b3cbfc084ce
7	7	student	V7_seed_default_department.xml	2026-05-14 16:55:11.549044	7	EXECUTED	9:cb5528bf1fe0fbfd31e0c440fff
8	8	student	V8_seed_projects.xml	2026-05-14 16:55:11.553608	8	EXECUTED	9:5207d8ffdaaf804ffff11482eb3

[illegible]

Inapoi

databasechangelog

CSV

WHERE ORDER BY

	id	author	filename	dateexecuted	orderexecuted	exectype	md5sum
1	1	student	V1_initial.xml	2026-05-14 16:55:11.518689	1	EXECUTED	9:9b788ded8582ba451ee24b18a662c
2	2	student	V2_add_phone.xml	2026-05-14 16:55:11.524532	2	EXECUTED	9:534583f99df3b62850d0eb188509d
3	3	student	V3_create_projects.xml	2026-05-14 16:55:11.532569	3	EXECUTED	9:1854251e4a110d412fe1437289b6c
4	4	student	V4_modify_salary.xml	2026-05-14 16:55:11.535148	4	EXECUTED	9:569dc028c440560d08f5c76790ad1
5	5	student	V5_indexes.xml	2026-05-14 16:55:11.540042	5	EXECUTED	9:31659271a79e804ecf102a2b2c210
6	6	student	V6_add_audit_columns.xml	2026-05-14 16:55:11.545978	6	EXECUTED	9:319b226c32b2f57b3c3bfc084ce9f
7	7	student	V7_seed_default_department.xml	2026-05-14 16:55:11.549044	7	EXECUTED	9:cb5528f8b1fe0fbfd31a0c440fb3b

Gestionarea conflictelor de locking optimist

```
// Simulare conflict optimistic: două load-uri și două save-uri
public void simulateOptimisticConflict(Integer id) { 1 usage
    Employee e1 = repo.findById(id).orElseThrow(() -> new RuntimeException("Employee not found: " + id));
    Employee e2 = repo.findById(id).orElseThrow(() -> new RuntimeException("Employee not found: " + id));

    e1.setEmail(e1.getEmail() + ".a");
    repo.save(e1); // increments version

    try {
        e2.setEmail(e2.getEmail() + ".b");
        repo.save(e2);
    } catch (ObjectOptimisticLockingFailureException ex) {
        throw new RuntimeException("Optimistic lock conflict detected (simulated).", ex);
    }
}
```

```
5 | bob@example.com | ver=0
4 | alice@example.com.a | ver=1

=== MENU ===
1) Create employee
2) Update employee
3) Simulate optimistic lock conflict
4) Soft delete employee
5) List employees
6) List deleted employees (admin)
7) Restore employee
8) Force delete (hard)
9) Exit
Choose: 3
Employee id: 4
```

Conflict: Optimistic lock conflict detected (simulated).

Operații de ștergere soft și restaurare

```
2026-05-14T17:26:16.487+03:00 DEBUG 57924 --- [main] org.hibernate.SQL
update
  employees
set
  created_at=?,
  created_by=?,
  deleted_at=?,
  deleted_by=?,
  department_id=?,
  email=?,
  is_deleted=?,
  phone=?,
  salary=?,
  updated_at=?,
  updated_by=?,
  version=?
where
  id=?
  and version=?
Soft deleted.
```

```
=== MENU ===
1) Create employee
2) Update employee
3) Simulate optimistic lock conflict
4) Soft delete employee
5) List employees
6) List deleted employees (admin)
7) Restore employee
8) Force delete (hard)
9) Exit
Choose: 5
2026-05-14T17:26:52.950+03:00 DEBUG 57924 --- [main] org.hibernate.SQL
select
  e1_0.id,
  e1_0.created_at,
  e1_0.created_by,
  e1_0.deleted_at,
  e1_0.deleted_by,
  e1_0.department_id,
  e1_0.email,
  e1_0.is_deleted,
  e1_0.phone,
  e1_0.salary,
  e1_0.updated_at,
  e1_0.updated_by,
  e1_0.version
from
  employees e1_0
where
  (
    e1_0.is_deleted = false
  )
5 | bob@example.com | ver=0
```

```

=== MENU ===
1) Create employee
2) Update employee
3) Simulate optimistic lock conflict
4) Soft delete employee
5) List employees
6) List deleted employees (admin)
7) Restore employee
8) Force delete (hard)
9) Exit
Choose: 7
Employee id: 4
2026-05-14T17:28:51.059+03:00 DEBUG 57924 --- [      main] org.hibernate.SQL:
    SELECT
    *
    FROM
        employees
    WHERE
        id = ?
2026-05-14T17:28:51.064+03:00 DEBUG 57924 --- [      main] org.hibernate.SQL:
    update
    employees
    set
        created_at=?,
        created_by=?,
        deleted_at=?,
        deleted_by=?,
        department_id=?,
        email=?,
        is_deleted=?,
        phone=?,
        salary=?,
        updated_at=?,
        updated_by=?,
        version=?
    where
        id=?
        and version=?
Restored.

```

```

=== MENU ===
1) Create employee
2) Update employee
3) Simulate optimistic lock conflict
4) Soft delete employee
5) List employees
6) List deleted employees (admin)
7) Restore employee
8) Force delete (hard)
9) Exit
Choose: 5
2026-05-14T17:29:10.634+03:00 DEBUG 57924 --- [      main] org.hibernate.SQL:
    select
        e1_0.id,
        e1_0.created_at,
        e1_0.created_by,
        e1_0.deleted_at,
        e1_0.deleted_by,
        e1_0.department_id,
        e1_0.email,
        e1_0.is_deleted,
        e1_0.phone,
        e1_0.salary,
        e1_0.updated_at,
        e1_0.updated_by,
        e1_0.version
    from
        employees e1_0
    where
        (
            e1_0.is_deleted = false
        )
5 | bob@example.com | ver=0
4 | alice@example.com.a.a | ver=4

```

